Практическая работа №10

Транзакции с успешным выполнением и откатом

Транзакция — это последовательность операторов SQL, выполняющихся как единая операция, которая не прерывается другими клиентами. То есть пока происходит работа с записями таблицы (их обновление или удаление), никто другой не может получить доступ к этим записям, т. к. СУБД MySQL автоматически блокирует доступ к ним.

Транзакции позволяют объединять операторы в группу и гарантировать, что все операции внутри группы будут выполнены успешно. Если часть транзакции выполняется со сбоем, результаты выполнения всех операторов транзакции до места сбоя отменяются, приводя базу данных к виду, в котором она была до выполнения транзакции. Оператор COMMIT сохраняет изменения на диске. Для того чтобы включить режим автоматического завершения транзакций только для отдельной последовательности операторов, можно воспользоваться оператором **START TRANSACTION**. Кроме того, существует ряд операторов, которые неявно завершают транзакцию, как если бы был вызван оператор **COMMIT**. К их числу относятся: ALTER TABLE, BEGIN, CREATE INDEX, CREATE TABLE, CREATE DATABASE, DROP DATABASE, DROP INDEX, DROP TABLE, DROP DATABASE, LOAD MASTER DATA, LOCK TABLES, RENAME, SET AUTOCOMMIT=1, START TRANSACTION, TRUNCATE TABLE.

Транзакции не могут быть вложенными. Это связано с тем, что любой оператор, начинающий транзакцию, приводит к завершению предыдущей транзакции.

Свойства транзакций:

1. Логическая неделимость (атомарность, Atomicity) означает, что выполняются либо все операции (команды), входящие в транзакцию, либо ни одной. Система гарантирует невозможность запоминания части изменений, произведённых транзакцией. До тех пор, пока транзакция не завершена, её можно "откатить", т.е. отменить все сделанные командами транзакции изменения. Успешное выполнение транзакции (фиксация) означает, что все команды транзакции проанализированы, интерпретированы как правильные и безошибочно исполнены.

2. Согласованность (Consistency): транзакция начинается на согласованном множестве данных и после её завершения множество данных согласовано. Состояние БД является согласованным, если данные удовлетворяют всем установленным ограничениям целостности и относятся к одному моменту в состоянии предметной области.

3. Изолированность (Isolation), т.е. отсутствие влияния транзакций друг на друга.

4. Устойчивость (Durability): результаты завершённой транзакции не могут быть потеряны. Возврат БД в предыдущее состояние может быть достигнут только путём запуска компенсирующей транзакции.

Управление журналом транзакций.

ib_logfile0 и ib_logfile1 - файлы журнала транзакций, все транзакции сначала помещаются сюда, затем перегоняются в файлы единого табличного пространства: если сервер MySQL останавливается штатно, все транзакции из журнала сохраняются в таблицу. Получить путь к каталогу, в котором размещены файлы транзакций, можно при помощи следующего запроса: **SHOW VARIABLES LIKE 'datadir';**

За режим управления сохранения транзакций отвечает переменная innodb_flush_log_at_trx_commit, которая может принимать следующие значения:

0 — сохранение журнала раз в секунду,

1 — сохранение после каждой транзакции,

2 — сохранение журнала раз в секунду и после каждой транзакции.

Узнать текущий режим сохранения транзакций: SHOW VARIABLES LIKE 'innodb_flush_log_at_trx_commit';

Установить новое значение: SET GLOBAL innodb_flush_log_at_trx_commit = 0;

К сожалению, на платном сервере это сделать невозможно, только на локальном хосте.

Пример 1. Использование COMMIT.

Ниже примеры, в которых осуществляются транзакции, затрагивающие выборку, обновление и вставку данных в одну и несколько таблиц.

Транзакция, иллюстрирующая обновление 3 поля в таблице «Пассажиры», а именно: возраст (чтобы не считать вручную, а поле автоматически заполнялось с помощью транзакции), комментарий (автоматически заполняется в зависимости от возраста), скидка на билет (заполняется в зависимости от возраста) В транзакции находится сразу 3 запроса, после того, как она выполнится будут обновлены данные в текущей таблице.

```
start TRANSACTION;
UPDATE passengers set age=timestampdiff(year, date_of_birth, now()) where age='' or age<0;
update passengers set comment=if(age>18,'совершеннолетний','совршеннолетний');
update passengers set discount=CASE
when age<2 then 100
when age>=2 and age<18 then 40
ELSE 0
end;
COMMIT
```

Рисунок 1 – Транзакция на обновление в рамках одной таблицы.

Транзакция, иллюстрирующая обновление данных в одной таблице, но с помощью значений, взятых из других таблиц, которые связаны с ней. Таблица «Регистрация» содержит поле «Стоимость», отвечающая за цену за билет для конкретного пассажира, который направляется на заданном рейсе. Для того, чтобы получить значение для этого поля необходимо соединить таблицы: «Регистрация» связана с «Пассажиры» по идентификатору пассажира, «Регистрация» и «Рейсы» по идентификатору рейса, «Города» и «Рейсы» по номеру города. Все эти данные нужны, чтобы в дальнейшем посчитать стоимость билета. Транзакция содержит один запрос, но при этом связывается с несколькими таблицами. Данные взяты из нескольких таблиц, что позволяет обеспечить целостность получаемых данных.

```
start transaction;
update register inner join passengers on register.id_passenger=passengers.id_pass
inner join avia on register.id_avia=avia.id_avia
inner join city on avia.id_city=city.id_city
set price = CASE
WHEN passengers.discount=40 THEN city.distance*4-city.distance*4*passengers.discount/100
WHEN passengers.discount=0 then 0
when passengers.discount=100 THEN city.distance*4
END;
COMMIT
```

Рисунок 2 – Транзакция на обновление в одной таблице с помощью данных из разных таблиц.

Стандартная транзакция, иллюстрирующая уменьшение количество товаров на складе после выборки по заданному товару.

```
start TRANSACTION;
select * from tovar where id_tovar=1;
update tovar set amount=amount -1;
commit;
```

Рисунок 3 – Транзакция на обновление после выборки

Вставка данных на основании выборки также возможна. На рейс регистрируются пассажиры на разные рейсы. При этом на один рейс может быть зарегистрировано много человек, как и один человек может улетать разными рейсами. В текущем примере показана транзакция, при которой данные в таблицу «Регистрация» поступают из таблицы

«Рейс», при этом значение рейса берется случайным образом. Таким образом, из запроса на выборку данные поступают в запрос на вставку. Действует это все в рамках одной транзкации.

```
start transaction;
INSERT INTO `register`( `id_avia`)
    select id_avia from avia where id_avia between 2 and 24 limit 1;
    commit
```

Рисунок 4 – Транзакция на вставку с помощью запроса на выборку из другой таблицы

Пример 2. Использование ROLLBACK

Создание стандартной транзакции отката. SET AUTOCOMMIT=0 позволяет отключить явный и неявный оператор COMMIT, TRUNCATE TABLE очищает содержимое таблицы, ROLLBACK препятствует этому выполнению. Стоит заметить, что выполнение этой транзакции невозможно, т.к. все таблицы в базе данных связаны, поэтому при очистке данных нарушается целостность.

```
start transaction;
set AUTOCOMMIT=0;
TRUNCATE table passengers;
ROLLBACK
```

Рисунок 5 – Создание отката для очистки данных из таблицы.

Описанная ниже транзакция создает откат при попытке выставить отрицательный возраст для любого из имеющихся пассажиров.

```
start transaction;
set AUTOCOMMIT=0;
UPDATE passengers set age=rand()*(-100) where id_pass between 2 AND 24;
ROLLBACK
```

Рисунок 6 – Создание отката при обновлении данных.

Задание.

Создать транзакции с использованием операторов COMMIT, ROLLBACK.

С использованием COMMIT:

2 транзакции на обновление для одной и нескольких таблиц;

2 транзакции на вставку с учетом выборки из одной и нескольких таблиц; 2 транзакции на выборку и обновление для одной и нескольких таблиц.

С использование ROLLBACK:

2 транзакции на откат обновления;

2 транзакции на откат удаления или очистки.

Совет: сохраняйте текст транзакций.